

Discussion 12

Note: Your TA will probably not cover all the problems on this worksheet. The discussion worksheets are not designed to be finished within an hour. They are deliberately made slightly longer so they can serve as resources you can use to practice, reinforce, and build upon concepts discussed in lectures, discussions, and homework.

This Week's Cool AI Demo/Video:

Gemini 3 video chat: https://play.google.com/store/apps/details?id=com.google.android.apps.bard&hl=en_US

Google's Cursor clone just launched: <https://antigravity.google/download>

Sunday came out of stealth: <https://www.youtube.com/watch?v=jj0fpsMRhL4>

1 Supervised Fine-tuning for Vision Language Models

A vision language model (VLM) is a model that integrates visual and textual data, enabling it to understand and process both images and text. By combining a vision encoder and a large language model (LLM), VLMs can perform tasks like generating captions for images, answering questions about an image, and even creating images from text descriptions.

To integrate a **Vision Encoder** that processes an image and outputs patch embeddings of dimension D_{vis} and an **LLM** that expects token embeddings of dimension D_{txt} , we typically define a **Projection Layer** (W_P) that projects visual embeddings into the text embedding space. Supervised Fine-tuning (SFT) is then performed with a small image-text dataset to align the visual features with the language model's embedding space.

- (a) **Projection Dimensions:** Suppose the Vision Encoder outputs an embedding H_{vis} of size $D_{vis} = 1024$ and the LLM expects embeddings of size $D_{txt} = 4096$. What must be the dimensions of the weight matrix, W_P , to project the visual features to the text space if we left-multiply the image embedding by the projection matrix?

Solution:

To project a vector of size 1024 to an output space of dimension 4096 via matrix multiplication, the dimensions must be 4096×1024 .

- (b) **Freezing Strategies:** During the initial alignment stage (and often during SFT), it is standard practice to keep the Vision Encoder and the LLM **frozen**, updating only the projection layer. Give two distinct reasons why we freeze the backbone models.

Solution:

- **Prevention of Catastrophic Forgetting:** The LLM and Vision Encoder are usually foundation models trained on very large datasets and are capable of generalizing to a wide range of tasks. Full fine-tuning on a smaller image-text dataset risks degrading their original generalization capabilities.
 - **Computational Efficiency:** Again, the LLM and Vision Encoder are usually foundation models, which typically have billions of parameters. Calculating gradients for billion parameter models is extremely memory and compute intensive. Freezing the weights of these models significantly reduces the VRAM requirements and speeds up training, as gradients are only computed for the small projection layer.
- (c) **Image Token Granularity:** The classify token ([CLS]) is a special token that represents the entire input sequence for a classification task. We can choose to project only the Vision Encoder’s classification token into the LLM embedding space, or we can project the entire sequence of patch tokens. What is the primary trade-off for each of these two approaches?

Solution:

- **Using [CLS] only:** This is **highly efficient**, in that only one token is added to the LLM context, but it results in significant **information loss**, particularly regarding spatial details (e.g., the token may capture the object in the images but no details about it).
- **Using Patch tokens:** This preserves fine-grained **spatial information**, allowing for detailed image description and grounding, but it consumes a large portion of the LLM’s **context window** (e.g., 256+ tokens per image) and increases inference latency.

While early models used a single image embedding, nearly all high-performance VLMs we have today keep the full sequence of patch tokens.

- (d) **The Role of SFT Data:** Explain why we cannot simply rely on the pre-training of the individual components and why the SFT stage (using Image-Text pairs) is necessary even if the embedding dimensions are the same.

Solution:

Even though the Vision Encoder understands images and the LLM understands text, their embedding spaces are initially disjoint. The numerical vector for a “cat” in the vision model does not naturally align with the token embedding for “cat” in the LLM. The SFT stage updates the projector to map visual features into the *same* semantic manifold as the text embeddings. Without this, the LLM would interpret the projected image tokens as noise.

2 Self-Supervised Learning

(a) Fill in the blanks in the following table.

Method Name	Input Type	Pretext Task	Generative or Discriminative	Loss Function
Autoencoder	Image	Reconstruct the input image.		
Context Encoder	Masked Image	Predict the missing content in the masked region of the input.		
Image Rotation	Rotated Image	Predict the rotation angle applied to the input (0° , 90° , 180° , or 270°).		
SimCLR (Contrastive Learning)	Two Images	Determine whether images are augmentations of the same image or different images.		

Solution:

Method Name	Input Type	Pretext Task	Generative or Discriminative	Loss Function
Autoencoder	Image	Reconstruct the input image.	Generative	Mean Squared Error (MSE)
Context Encoder	Masked Image	Predict the missing content in the masked region of the input.	Generative	MSE + Adversarial Loss
Image Rotation	Rotated Image	Predict the rotation angle applied to the input (0° , 90° , 180° , or 270°).	Discriminative	Cross-Entropy
SimCLR (Contrastive Learning)	Two Images	Determine whether images are augmentations of the same image or different images.	Discriminative	Contrastive Loss

(b) When training **context encoders**, we often use a joint loss function:

$$\mathcal{L} = \mathcal{L}_{\text{rec}} + \lambda \mathcal{L}_{\text{adv}}$$

where:

- $\mathcal{L}_{\text{rec}}$ is the reconstruction loss (e.g., MSE) applied to the masked region
 - $\mathcal{L}_{\text{adv}}$ is the adversarial loss from a discriminator
 - λ is a weighting factor that balances the two terms
- (i) If we trained using **only** $\mathcal{L}_{\text{rec}}$, what artifact might we observe in the generated missing region? Why?

Solution:

The inpainted region would likely appear **blurry**. Minimizing

$$\mathcal{L}_{\text{rec}} = \|M \odot (x - \hat{x})\|_2^2$$

corresponds mathematically to averaging all possible valid completions. When multiple completions are plausible, the average produces a smooth, blurry output.

- (ii) Why does adding the adversarial loss $\mathcal{L}_{\text{adv}}$ help fix this issue?

Solution:

The adversarial loss encourages the generator to produce outputs that the discriminator classifies as **realistic**. This forces the network to select one **sharp, plausible mode** of the missing region rather than averaging over all possibilities, reducing blurriness.

3 Autoregressive vs. Diffusion Models

Autoregressive (AR) models, such as next-word prediction GPT models, and Diffusion models are two popular categories of deep generative models. They both achieve high-quality generation by decomposing the process into many steps. In this problem, we explore the design choices that lead to their training efficiency and their distinct generation mechanisms.

- (a) **Computational Graph and Training Efficiency:** Explain the core design principle that allows AR and diffusion models to be training-efficient despite performing a deep, sequential computation at inference time.

Solution:

The efficiency stems from the decoupling of the deep inference path from the training step's backpropagation. Both models optimize a local, single-step objective during training, avoiding the calculation of gradients through the entire, unrolled generative sequence.

During training, an AR model predicts x_n based on $x_{<n}$. The loss gradient is calculated only for that single prediction step, $p(x_n|x_{<n})$, avoiding backpropagation through all previous elements.

Similarly, a diffusion model is trained to predict the noise ϵ at a single, random time step t . The loss requires gradients only through a single pass of the noise-prediction network.

For both modeling techniques, the deep, sequential computational path (many dependent steps) is only constructed during generation (inference), where gradients are not needed.

- (b) **Frequency/Scale Generation in Diffusion Models:** Describe the typical pattern of frequency content (or scale of details) that is synthesized as the generative (denoising) process moves from high noise ($t = T$) toward the clean image ($t = 0$).

Solution:

Diffusion models typically synthesize image content in a coarse-to-fine manner, generating low-frequency structures first and high-frequency details later.

- **Early Steps ($t \approx T$):** The model primarily generates low-frequency content, corresponding to large-scale structures and the global layout. High noise variance at these steps masks fine details, so the model focuses on establishing broad features.
- **Late Steps ($t \approx 0$):** The model progressively adds high-frequency content, refining textures, edges, and small details. As the noise level decreases, the model can make precise adjustments to the existing global structure, producing the final detailed image.

This coarse-to-fine progression allows the generative process to be stable and coherent, ensuring that the global structure is in place before fine details are synthesized.